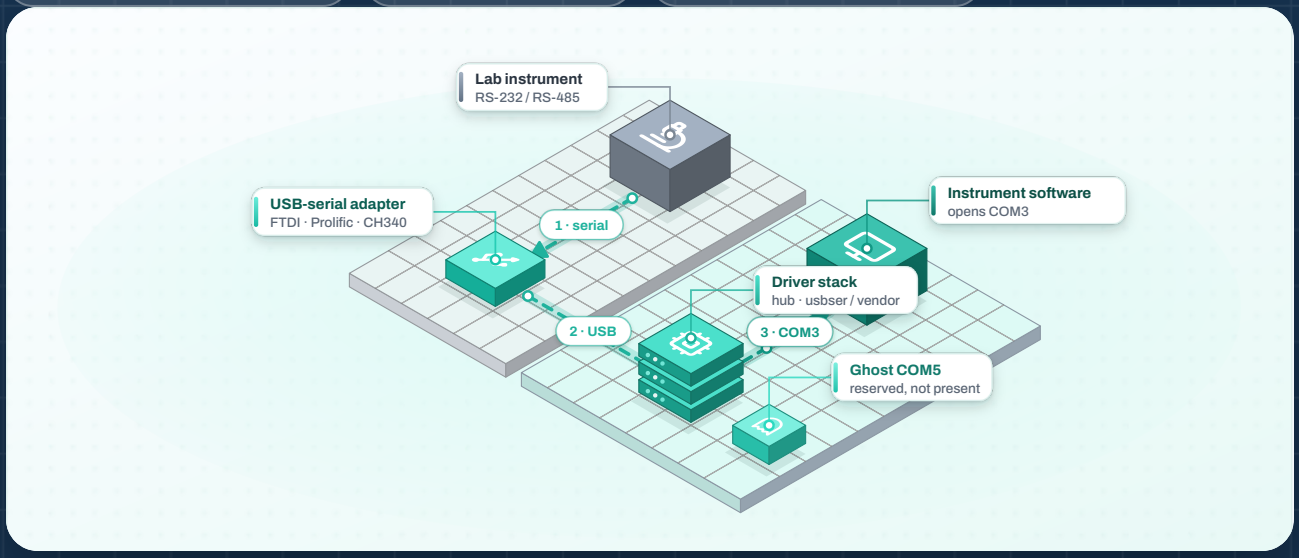
STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

**SOP-WIN-001**

# Troubleshooting USB Serial (COM Port) Device Connectivity on Windows

Diagnose and repair USB-to-serial adapters and COM port assignments five ways: Device Manager, a CMD repair sequence, PowerShell with pnputil, a power-management fix, and Intune Remediations.

Windows Desktop Support · Devices and Drivers · Endpoint Troubleshooting




| | |
|---|---|
| DOCUMENT ID | SOP-WIN-001 |
| VERSION | 2.0 |
| EFFECTIVE | October 2026 |
| AUDIENCE | Desktop support (Tier 1–3), lab and instrument support engineers, endpoint administrators |
| AUTHOR | Richmond Dieujuste, Principal Consultant |

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition — client identifiers removed

# Contents

**1. Concepts You Need First** 3

1.1 How a USB-to-serial adapter becomes COM3 3

1.2 Device Manager and problem codes 5

1.3 Ghost COM ports 6

1.4 When the operating system itself is the problem: SFC versus DISM 7

1.5 Windows Update components and driver installation 8

1.6 USB selective suspend 9

1.7 Choosing a method 10

**2. Access, Prerequisites and Planning** 12

2.1 Level of access required 12

2.2 Prerequisites checklist 13

2.3 Risk and planning notes 13

**3. Method A · Device Manager (GUI)** 14

**4. Method B · Command Prompt Repair Sequence** 16

**5. Method C · PowerShell and pnputil** 18

5.1 Inventory ports and problem codes 18

5.2 Restart a port, remove ghosts, reinstall a driver 18

5.3 Base template · scripted repair with evidence 19

**6. Methods D and E · Power Management and Enterprise Options** 21

6.1 Method D · Per device: Device Manager 21

6.2 Method D · Power plan: powercfg 21

6.3 Method D · Machine-wide: registry 22

6.4 Method E · Remote PowerShell for one or a few PCs 22

6.5 Method E · Intune Remediations for a fleet 22

6.6 Method E · Deploying the right driver centrally 23

**7. Verification, Rollback and Troubleshooting** 24

7.1 Verification checklist 24

7.2 A quick loopback test 24

7.3 Evidence for the ticket 24

7.4 Rollback by action 25
7.5 Troubleshooting flowchart 25
7.6 Symptom, cause and fix 26
7.7 When to escalate 26
**8. Summary, Glossary and Review 27**
8.1 Chapter summary 27
8.2 Glossary 27

## LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Explain how a USB-to-serial adapter becomes a COM port: the USB driver stack, the Ports class and the COM Name Arbiter.
- Read Device Manager problem codes and decide whether the fault is cabling, driver, OS integrity or power management.
- Explain the difference between SFC and DISM, and why resetting SoftwareDistribution and catroot2 helps stuck driver installs.
- Find and remove ghost COM ports and reassign a COM number that an application expects.
- Disable USB selective suspend with powercfg, Device Manager or the registry, and know when that is justified.
- Automate detection and repair across a fleet with PowerShell, pnputil and Intune Remediations.

| TIME | DIFFICULTY | MINIMUM ACCESS | RISK |
|---|---|---|---|
| 15 min (driver/port) to 45 min (OS repair) | Beginner to intermediate | Local administrator on the workstation | Low; DISM/SFC and WU reset are non-destructive |

CHAPTER 1

# Concepts You Need First

Laboratory instruments, barcode scanners, label printers, PLCs, network switches and medical test rigs still talk over **serial** (RS-232 or RS-485) links: a protocol older than the PC itself. Modern laptops and desktops have no physical serial port, so the link reaches Windows through a **USB-to-serial adapter**, either a separate cable or a chip built into the instrument. Windows presents that adapter to software as a **COM port** (COM3, COM4…), and the instrument software is configured to open one specific COM number.

When "the instrument stopped talking to the PC", the fault can sit in any of five places: the cable or USB port, the adapter driver, the COM number assignment, the operating system's ability to install and verify drivers, or USB power management. This SOP teaches you to tell those apart quickly and then gives you several ways to fix each one.

> **KEY CONCEPT · A COM PORT IS A NAME, NOT A SOCKET**
>
> Think of a COM number as a seat number on a train. The adapter is the passenger. Windows hands out seat numbers from a single register (the *COM Name Arbiter*), and the instrument software has a ticket that says "COM3". If the passenger boards at a different door (a different USB port) and Windows does not recognise them, they may be given seat COM5, while COM3 stays reserved for a passenger who is no longer on the train. The software then waits at COM3 forever. Most "lost connection" tickets are seat-number problems, not broken hardware.

## 1.1 How a USB-to-serial adapter becomes COM3

When you plug the adapter in, a chain of drivers loads, each one built on the one below. Device Manager only shows you the top of this stack, but knowing the layers tells you where to look.

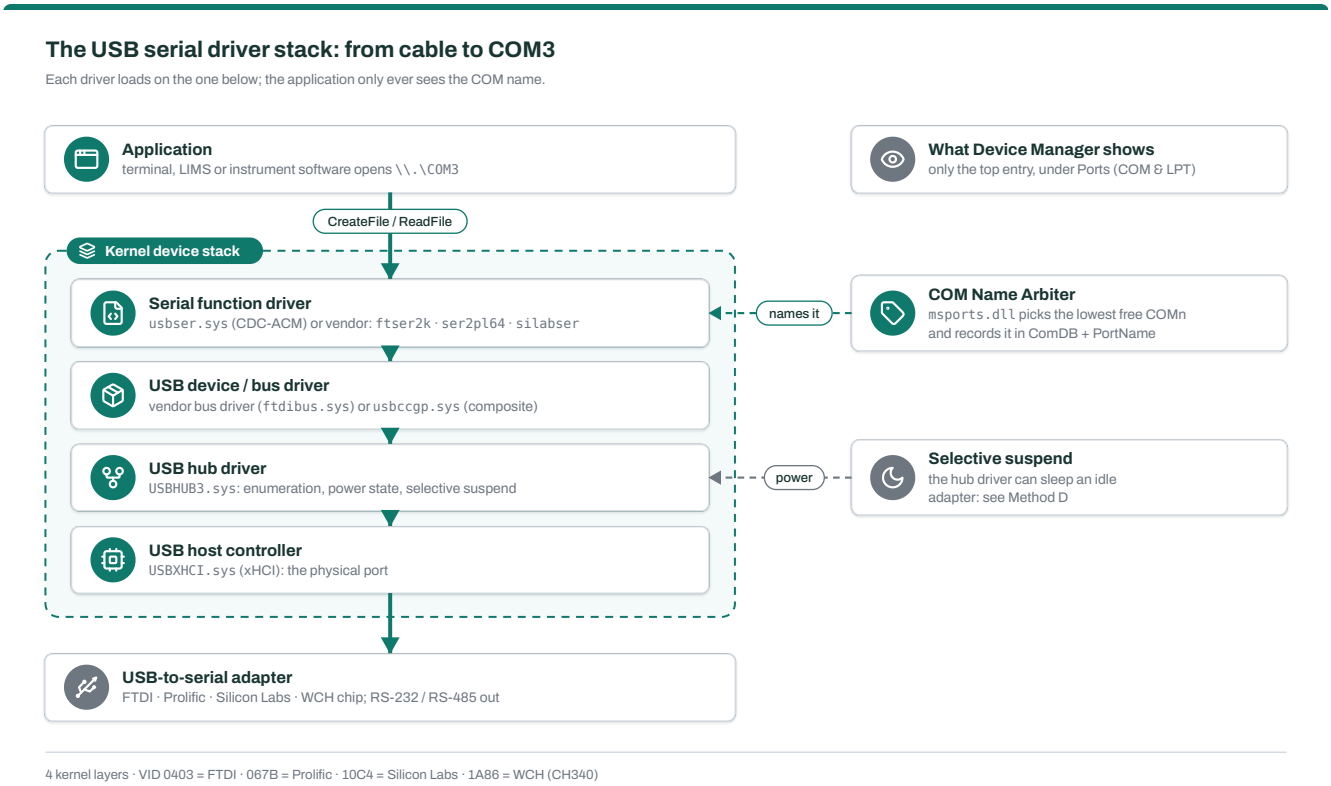



**Figure 1.** Device Manager shows only the top of the stack, so a fault in any lower layer surfaces as a problem with the COM port itself.

| Layer | What it does | What goes wrong |
|---|---|---|
| **Host controller and hub** | `USBXHCI.sys` and `USBHUB3.sys` enumerate the device, read its vendor and product IDs (VID/PID) and manage its power state. | Faulty port or hub, insufficient power, selective suspend putting the device to sleep. |
| **USB device / bus driver** | Vendor-specific (for example `ftdibus.sys` for FTDI chips) or the generic composite driver `usbccgp.sys`. | Wrong or unsigned driver, counterfeit chips rejected by newer drivers. |
| **Serial function driver** | Exposes a serial port. Windows 10/11 include `usbser.sys` for standard CDC-ACM devices; FTDI, Prolific (`ser2pl64.sys`), Silicon Labs (`silabser.sys`) and WCH (CH340) chips need vendor drivers. | Driver missing (device sits under *Other devices*), driver fails to start (Code 10). |
| **Ports class / COM Name Arbiter** | Assigns the lowest free COM number, records it in the *ComDB* bitmap and in the device's `PortName` registry value. | Ghost reservations, number changes after moving USB ports, two devices claiming one number. |

| | | |
|---|---|---|
| **Application** | Opens `\\.\COM3` with fixed baud rate, data bits, parity, stop bits and flow control. | Configured for the wrong COM number or line settings; another process already has the port open. |

**WORKED EXAMPLE · READING A HARDWARE ID**

In Device Manager, **Properties › Details › Hardware Ids** shows a string such as `USB\VID_0403&PID_6001&REV_0600`. The VID tells you the chip vendor: `0403` = FTDI, `067B` = Prolific, `10C4` = Silicon Labs, `1A86` = WCH (CH340). Now you know which vendor driver to install without opening the instrument.

## 1.2 Device Manager and problem codes

Device Manager (`devmgmt.msc`) is the window onto the Plug and Play (PnP) manager. A yellow triangle means the PnP manager recorded a **problem code** for that device, visible on the General tab of its properties. Learn the five that matter for serial adapters:

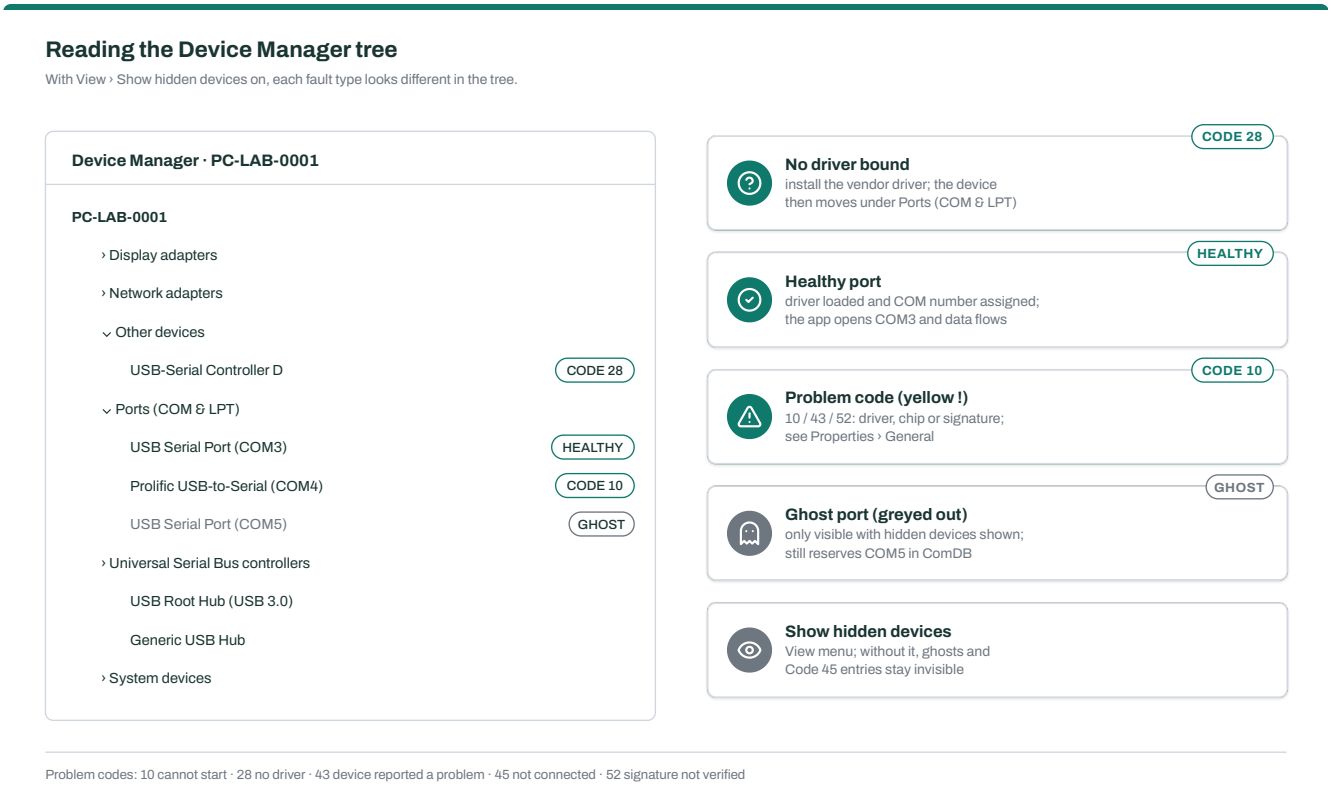



**Figure 2.** Turn on hidden devices first: a ghost reservation is invisible otherwise, yet it still blocks the COM number the application expects.

| Code | Message (abridged) | Usual meaning for a serial adapter |
|---|---|---|
| **Code 10** | This device cannot start. | Driver loaded but the chip refused to start. Classic with clone/counterfeit Prolific or FTDI chips and new drivers, or a damaged adapter. |
| **Code 28** | The drivers for this device are not installed. | No matching driver in the driver store. Install the vendor package. |
| **Code 43** | Windows has stopped this device because it has reported problems. | The device or its firmware returned an error. Try another port or cable, then reinstall the driver. |
| **Code 45** | Currently, this hardware device is not connected. | A non-present ("ghost") device. Expected for unplugged hardware. |
| **Code 52** | Windows cannot verify the digital signature for the drivers. | Unsigned driver, or the catalog database that verifies signatures is damaged (see catroot2 below). |

## 1.3 Ghost COM ports

Windows identifies a USB device by an *instance ID*. If the adapter's chip carries a unique serial number (most genuine FTDI chips do), the instance ID follows the adapter to any USB port and the COM number stays the same. If the chip has *no* serial number, the instance ID is built from the physical port location, so the same adapter in a different USB socket becomes a *new* device and receives a *new* COM number. The old device is now "non-present", but its COM number stays reserved in ComDB. Those reservations are called **ghost ports**, and they are why a workstation that has had many instruments attached ends up at COM17.

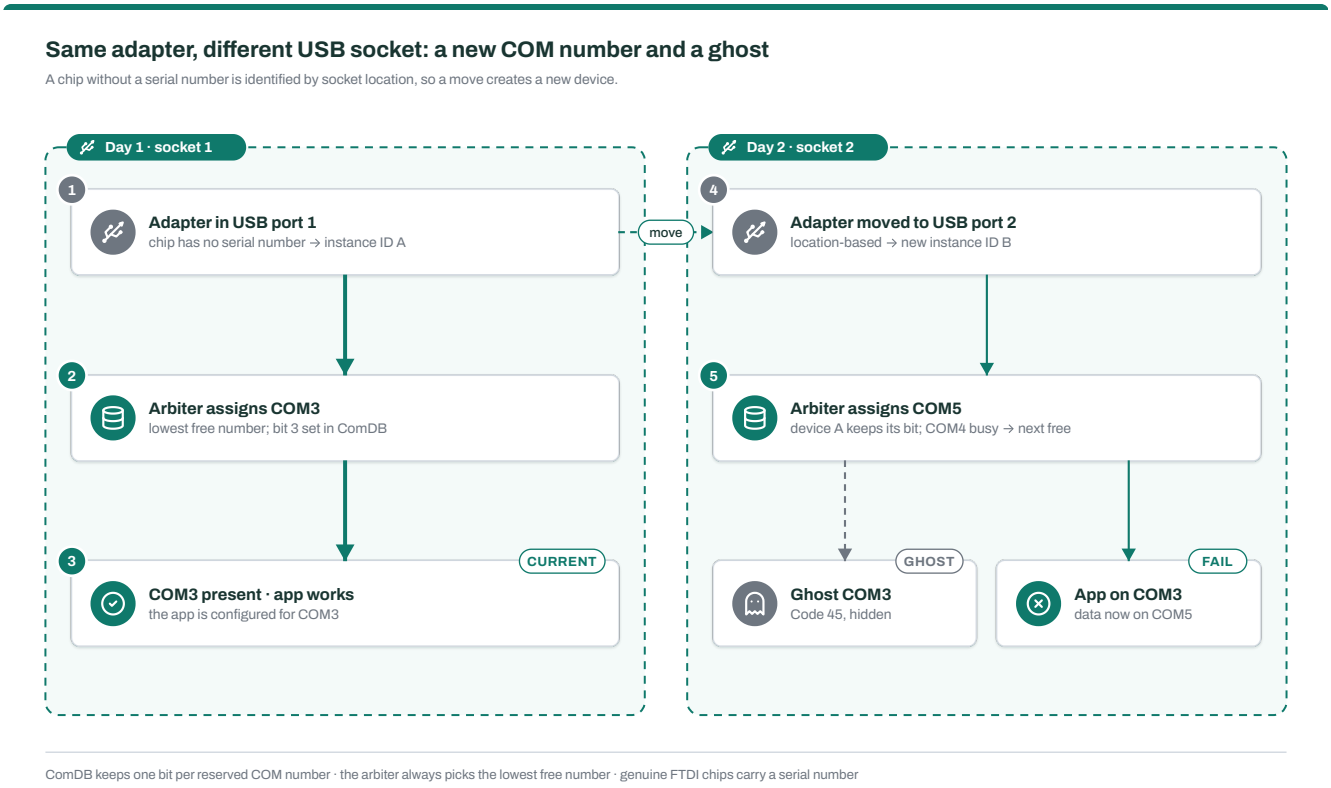



**Figure 3.** When an adapter has no serial number, keep it in the same USB socket or the COM number will change and leave a ghost behind.

## 1.4 When the operating system itself is the problem: SFC versus DISM

Sometimes the adapter, cable and driver package are fine, but Windows cannot install or start the driver because its own files are damaged. Two built-in tools repair that, and they work at different layers:

DISM · REPAIRS THE STORE

**Deployment Image Servicing and Management** checks and repairs the *component store* (`C:\Windows\WinSxS`), the master library of every Windows file version. `/RestoreHealth` downloads clean copies from Windows Update (or a mounted install image) to fix the library itself.

SFC · REPAIRS THE SYSTEM

**System File Checker** compares protected system files in `C:\Windows\System32` and elsewhere against the component store and replaces any that do not match. It can only be as good as the store it copies from.

**WHY · DISM FIRST, THEN SFC**

If the library is damaged, SFC copies damaged books. Run `DISM /RestoreHealth` first to fix the store, then `sfc /scannow` to fix the live files from the now-healthy store. The original field procedure ran SFC alone; it worked because the store was intact, but the DISM-first order is the reliable default. Logs: `%windir%\Logs\DISM\dism.log` and `%windir%\Logs\CBS\CBS.log`.

## 1.5 Windows Update components and driver installation

Driver installation shares plumbing with Windows Update. Two folders matter:

- **`C:\Windows\SoftwareDistribution`**: the Windows Update download cache and its database (*DataStore*). Drivers offered through Windows Update, and Windows Update's record of what is installed, live here. A corrupt database can make driver searches hang or fail with obscure errors.
- **`C:\Windows\System32\catroot2`**: the catalog database used by the *Cryptographic Services* (`CryptSvc`) service. Every signed driver ships with a `.cat` catalog file containing hashes of its files; Windows checks those hashes against this database before loading the driver. If catroot2 is damaged, perfectly good drivers fail signature checks (Code 52) or installs stall at "Installing device software".

You cannot rename those folders while the services that hold them are running, so the reset is always *stop services → rename → start services*. Windows rebuilds both folders empty on next use. Nothing of value is lost: the cost is that Windows Update history in Settings appears empty and pending downloads restart.

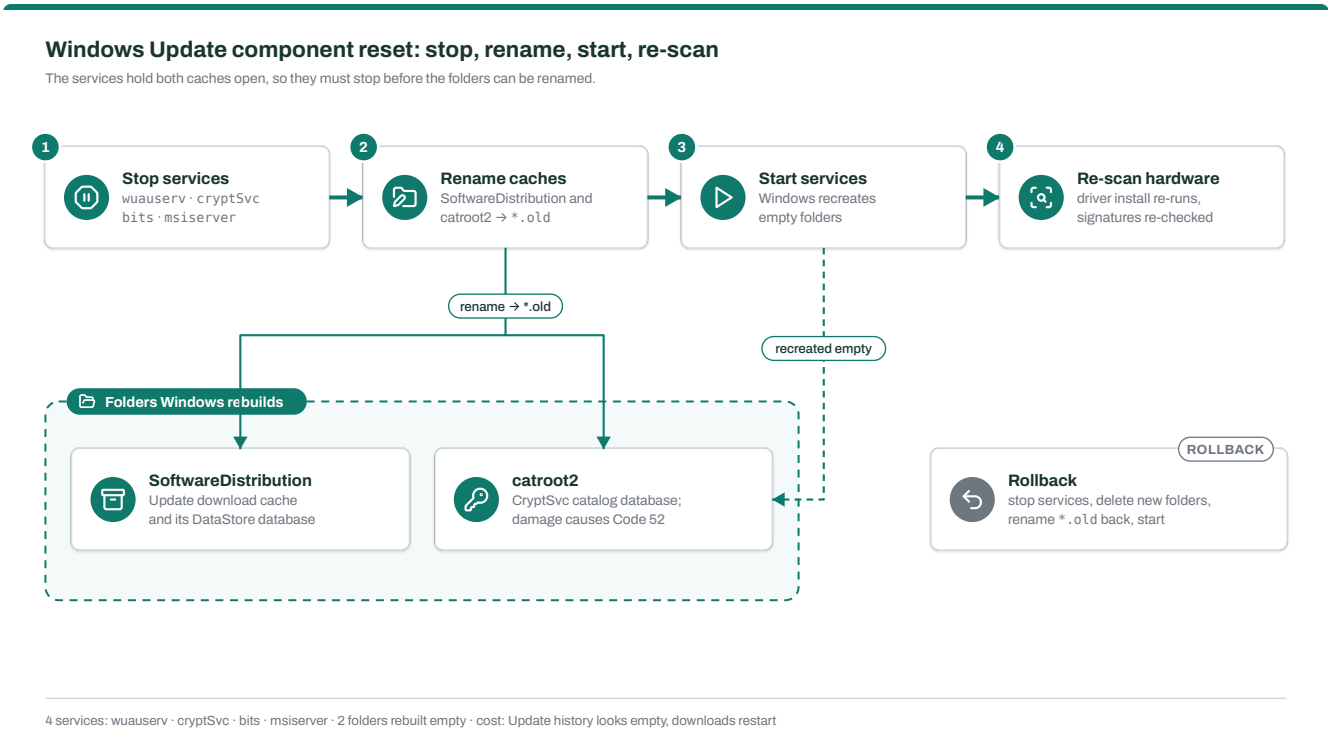



**Figure 4.** Nothing of value is lost in the reset; Windows rebuilds both caches, and renaming instead of deleting keeps a rollback path.

## 1.6 USB selective suspend

**Selective suspend** lets the USB hub driver put an idle device into a low-power state while the rest of the PC keeps running. It saves battery, but many serial adapters and instruments do not wake cleanly: the COM port stays listed, but data stops after the PC has been idle, or the port disappears and returns with a new number. If the symptom is "works after reboot, fails after lunch", suspect power management before drivers.

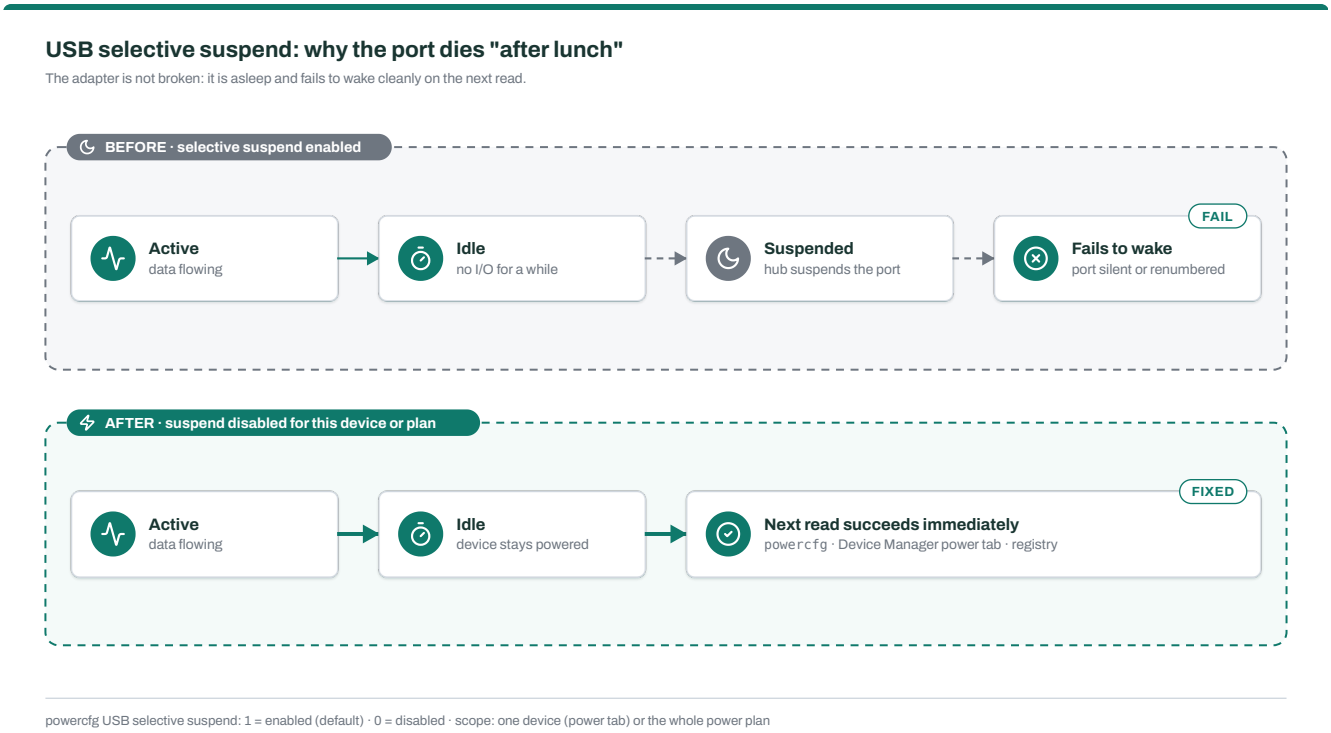



**Figure 5.** If a port works after a reboot but fails after idle time, fix power management before touching the driver.

## 1.7 Choosing a method

### A · Device Manager

Inspect, reinstall drivers, show hidden devices, reassign COM numbers.

BEST FOR: HANDS-ON TRIAGE

### B · CMD repair sequence

DISM, SFC and the Windows Update component reset from an elevated prompt.

BEST FOR: DRIVER INSTALLS THAT FAIL

### C · PowerShell + pnputil

Query ports and problem codes, remove ghosts, rescan, scripted reset.

BEST FOR: REPEATABLE, REMOTE FIXES

### D · Power management

Disable selective suspend with powercfg, Device Manager or the registry.

BEST FOR: DROPS AFTER IDLE OR SLEEP

### E · Intune / remote

Remediation scripts and Invoke-Command across many lab PCs.

BEST FOR: FLEETS OF INSTRUMENT PCS

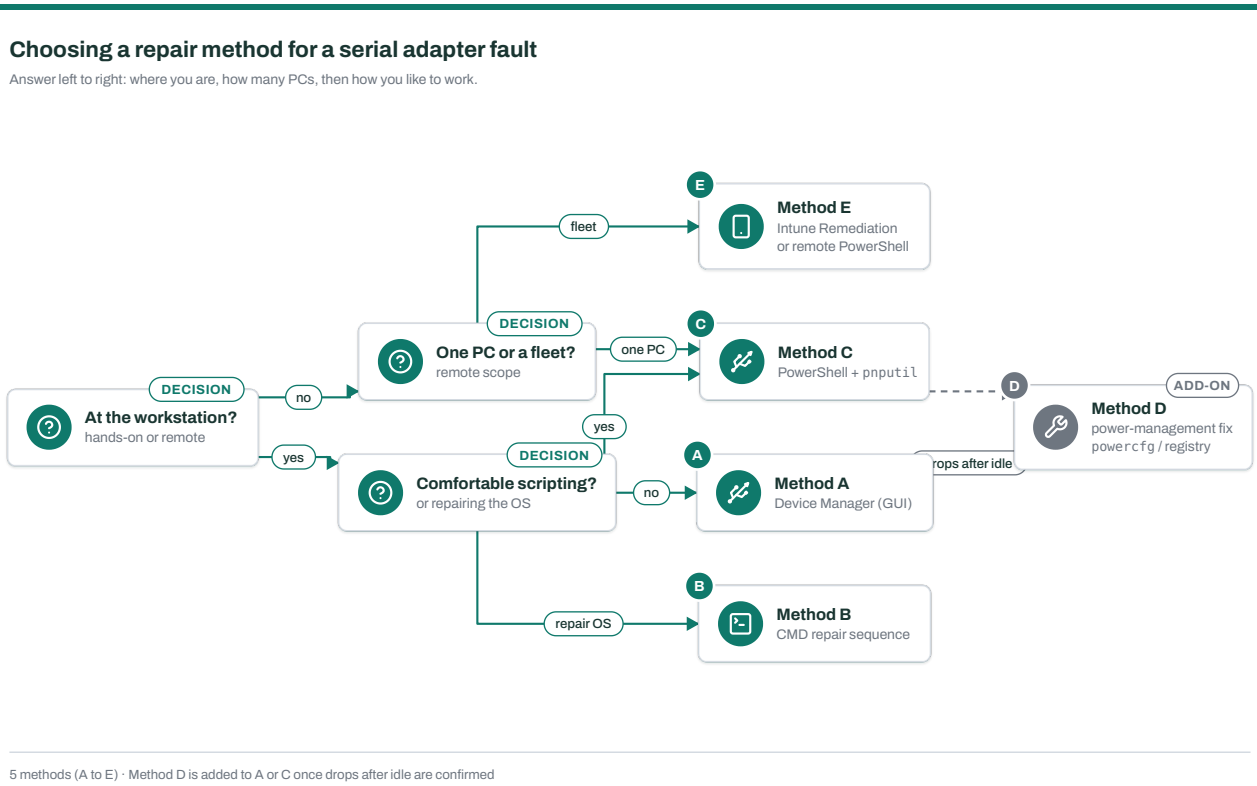



**Figure 6.** Pick the method that matches your access and scale; Method D is added to A or C once the root cause is known.

CHAPTER 2

# Access, Prerequisites and Planning

## 2.1 Level of access required

Viewing Device Manager needs no special rights, but every *change* in this SOP (installing drivers, removing devices, repairing system files, stopping services, changing power settings) requires local administrator rights on the workstation. Use a managed, rotated local administrator credential rather than a shared one.

| Task | Minimum access | Notes |
|---|---|---|
| **View devices, problem codes, hardware IDs** | Standard user | Device Manager opens read-only and warns that changes need administrator rights. |
| **Install, update, roll back or uninstall a driver** | Local Administrators group | Standard users can be allowed to install specific device classes with the policy **Computer Configuration › Administrative Templates › System › Driver Installation › Allow non-administrators to install drivers for these device setup classes.** |
| **DISM, SFC, stop/start services, rename system folders** | Local administrator, elevated prompt | Running without elevation fails with "You must be an administrator…". |
| **pnputil remove/add drivers and devices** | Local administrator, elevated | pnputil changes the driver store; treat as a change. |
| **Change power plan or registry** | Local administrator | `powercfg /setacvalueindex` on the active scheme needs elevation when the plan is enforced by policy. |
| **Remote PowerShell (Invoke-Command)** | Administrator on the target + WinRM enabled | Member of local Administrators or Remote Management Users with a JEA endpoint. |
| **Intune Remediations** | Intune Administrator, or a custom Intune role with *Device configurations* permissions; *Remote tasks › Run remediation* for on-demand runs | Requires Windows Enterprise/Education E3/E5 (or equivalent) licensing for devices; scripts run as SYSTEM by default. |

**GOOD PRACTICE · JUST-IN-TIME LOCAL ADMIN**

Retrieve the device's Windows LAPS password from Microsoft Entra ID or Active Directory for the session instead of using a shared technician account, and let LAPS rotate it afterwards. If device installation is blocked by policy (**System › Device Installation › Device Installation Restrictions**), the fix belongs to the policy owner, not to a local workaround.

## 2.2 Prerequisites checklist

- [ ] The ticket or change record, the instrument name and the COM number its software expects.
- [ ] The adapter's vendor (from the VID) and the vendor's current signed driver package, staged locally.
- [ ] A known-good USB cable and, if possible, a known-good spare adapter for swap testing.
- [ ] The instrument software closed; only one process can hold a COM port open.
- [ ] Confirmation that no run or acquisition is in progress on the instrument, and a maintenance window if a reboot is likely.
- [ ] Free disk space of at least 5 GB on C: for DISM to stage repair files.

## 2.3 Risk and planning notes

| Action | Risk | Notes |
|---|---|---|
| **Device Manager inspection, pnputil enumeration** | READ-ONLY | No change. |
| **DISM /RestoreHealth, sfc /scannow** | LOW | Replaces only damaged OS files. Takes 10–30 minutes; avoid on a PC acquiring data. |
| **Windows Update component reset** | MODERATE | Update history view resets and pending downloads restart. Keep the `.old` folders until verified. |
| **Uninstall driver / remove ghost device** | MODERATE | COM numbers may change; document the current mapping first. |
| **Delete driver package from store** | HIGHER | Other adapters of the same vendor lose their driver until reinstalled. |

CHAPTER 3

# Method A · Device Manager (GUI)

Device Manager is the fastest way to see which layer is failing. Work top-down: is the device present, does it have a driver, does it have the right COM number, does it stay awake?

## STEP A1 Triage the device

1. Close the instrument software. Press Win+X and select **Device Manager**, or run `devmgmt.msc`.
2. Expand **Ports (COM & LPT)**. Note every entry and its COM number.
3. Unplug the adapter and watch which entry disappears; plug it back in and watch it return. This identifies the right device without guesswork.
4. If nothing appears under Ports, expand **Other devices** and **Universal Serial Bus controllers**. An entry such as *USB-Serial Controller* with a yellow triangle means the driver is missing (Code 28).
5. If nothing changes anywhere when you plug in, the fault is physical: try another USB port directly on the PC (not a hub), another cable, then a spare adapter.

## STEP A2 Read the problem code and hardware ID

1. Right-click the device › **Properties**. The **General** tab shows *Device status* and any code.
2. On **Details**, choose **Hardware Ids** and note the VID and PID.
3. On **Driver**, note the provider, date and version. Compare with the vendor's current release.
4. On **Events**, review the most recent entries: "Device not migrated" or "Device install requested" events show when Windows last changed the device.

## STEP A3 Reinstall or roll back the driver

1. **Code 28**: run the vendor installer, or right-click › **Update driver** ›

`Browse my computer for drivers` and point to the extracted package folder.

2. **Code 10 or 43 after a recent update**: `Driver` tab › `Roll Back Driver` (available only if a previous version exists).
3. **Persistent Code 10/43**: right-click › `Uninstall device`, tick `Attempt to remove the driver for this device` only if you have the package ready, then `Action › Scan for hardware changes`.
4. **Code 52**: the signature check failed. Do not disable signature enforcement. Use Method B to repair catroot2 and the component store, then reinstall.

**WATCH OUT · COUNTERFEIT ADAPTER CHIPS**

Low-cost cables sometimes carry cloned Prolific or FTDI chips. Current vendor drivers detect many clones and refuse to start them (Code 10, or "PL2303HXA PHASED OUT" in the device name). Rolling back to an old driver is a temporary workaround at best; the reliable fix is a genuine adapter.

## STEP A4 Remove ghost ports and reassign the COM number

1. Select `View › Show hidden devices`. Non-present devices appear greyed out under `Ports (COM & LPT)`.
2. Right-click each greyed-out port that belongs to hardware no longer in use › `Uninstall device`. This releases its COM number.
3. Open the live port's `Properties` › `Port Settings` › `Advanced...` and set `COM Port Number` to the number the instrument software expects. (The dialog layout varies by vendor driver.)
4. If Windows warns the number is "in use", a ghost still holds it. Remove the ghost first rather than forcing the assignment.
5. Unplug and replug the adapter to confirm the number sticks. For adapters without a serial number, label the USB socket so users always use the same one.

CHAPTER 4

# Method B · Command Prompt Repair Sequence

Use this method when the driver install itself fails (Code 52, installs that hang, vendor installers that roll back) or when other devices on the same PC are also misbehaving. It repairs the operating system underneath the driver. This is the sequence from the reference engagement, with DISM added in front and the Windows Update service added to the reset.

## STEP B1 Repair the component store, then system files

1. Press Win, type `cmd`, right-click **Command Prompt** › **Run as administrator**. The title bar must read *Administrator*.
2. Check, then repair, the component store. `/CheckHealth` is instant; `/RestoreHealth` takes 10–20 minutes and may pause at 20% or 62% for a while; that is normal.
3. Run SFC and wait for the result (10–15 minutes).
4. Read the outcome. "Did not find any integrity violations" or "found corrupt files and successfully repaired them" are both good. "Found corrupt files but was unable to fix some of them" means repeat DISM with a known-good source.

ELEVATED COMMAND PROMPT · OS INTEGRITY — CMD

```
DISM /Online /Cleanup-Image /CheckHealth
DISM /Online /Cleanup-Image /RestoreHealth
sfc /scannow

:: No Windows Update access? Use matching install media (D:). Find the index with DISM /Get-WimInfo
DISM /Online /Cleanup-Image /RestoreHealth /Source:WIM:D:\sources\install.wim:1 /LimitAccess
```

## STEP B2 Reset the Windows Update components

1. Stop the four services: `wuauserv` (Windows Update, holds SoftwareDistribution), `cryptSvc` (Cryptographic Services, holds catroot2), `bits` (downloads payloads) and `msiserver` (Windows Installer, clears a stuck MSI-based driver install).
2. Rename the two cache folders. If `ren` reports "Access is denied", a service is still running; repeat the stop.
3. Start the services again.
4. Reconnect the adapter and reinstall the driver (Method A, Step A3).

ELEVATED COMMAND PROMPT · WINDOWS UPDATE COMPONENT RESET CMD

```
net stop wuauserv
net stop cryptSvc
net stop bits
net stop msiserver

ren C:\Windows\SoftwareDistribution SoftwareDistribution.old
ren C:\Windows\System32\catroot2 catroot2.old

net start wuauserv
net start cryptSvc
net start bits
net start msiserver
```

If a `.old` folder already exists from an earlier reset, `ren` fails: use a dated suffix such as `.old-20261015`.

## STEP B3 Re-detect hardware and verify

1. Trigger a hardware rescan without opening Device Manager: `pnputil /scan-devices`.
2. List the ports and their status: `pnputil /enum-devices /class Ports`.
3. Check the event log for driver install results: **Event Viewer › Applications and Services Logs › Microsoft › Windows › Kernel-PnP › Configuration.**
4. If the port is healthy, open the instrument software and test a read. If issues persist, restart the workstation (about 5 minutes) and retest.

QUICK CHECK AFTER THE RESET CMD

```
reg query HKLM\HARDWARE\DEVICEMAP\SERIALCOMM
```

**KEY CONCEPT · THE SERIALCOMM MAP**

`HKLM\HARDWARE\DEVICEMAP\SERIALCOMM` is rebuilt at every boot and every time a serial port starts. Each value maps a kernel device name (for example `\Device\VCP0` or `\Device\USBSER000`) to a COM name. If your port is missing from this key, its driver is not running, whatever Device Manager's tree suggests.

CHAPTER 5

# Method C · PowerShell and pnputil

PowerShell turns the Device Manager checks into objects you can filter, log and repeat, and it works over a remote session. The `PnpDevice` module (built into Windows 10/11) and `pnputil.exe` cover nearly everything Device Manager does.

## 5.1 Inventory ports and problem codes

INVENTORY (READ-ONLY) POWERSHELL

```powershell
#  All serial ports, present and non-present, with COM name and problem code
Get-PnpDevice -Class Ports |
  Select-Object Status, Present, FriendlyName, ConfigManagerErrorCode, InstanceId |
  Sort-Object Present -Descending | Format-Table -AutoSize

#  The COM name exactly as the driver registered it
Get-PnpDevice -Class Ports -PresentOnly | ForEach-Object {
  $p = Get-ItemProperty "HKLM:\SYSTEM\CurrentControlSet\Enum\$($_.InstanceId)\Device Parameters" -
ErrorAction SilentlyContinue
  [pscustomobject]@{ Port = $p.PortName; Device = $_.FriendlyName; Vid = ($_.InstanceId -replace
'.*VID_([0-9A-F]{4}).*','$1') }
}

#  What the running system can open right now
[System.IO.Ports.SerialPort]::GetPortNames()

#  Devices with a driver problem anywhere (yellow triangles)
Get-PnpDevice -PresentOnly | Where-Object ConfigManagerErrorCode -ne 0 |
  Select-Object Class, FriendlyName, ConfigManagerErrorCode
```

The same inventory with `pnputil` (Windows 11 and recent Windows 10 builds) is useful from a plain Command Prompt or a recovery shell:

PNPUTIL EQUIVALENTS CMD

```cmd
pnputil /enum-devices /class Ports                &:: present ports
pnputil /enum-devices /class Ports /disconnected  &:: ghost ports
pnputil /enum-devices /problem                    &:: any device with a problem code
pnputil /enum-devices /class Ports /drivers       &:: which oemNN.inf each port uses
pnputil /enum-drivers /class Ports                &:: driver packages in the store
```

## 5.2 Restart a port, remove ghosts, reinstall a driver

TARGETED REPAIRS (ELEVATED) POWERSHELL

```powershell
#  Restart (disable + enable) a misbehaving port without unplugging it
$port = Get-PnpDevice -Class Ports -PresentOnly | Where-Object FriendlyName -like '*(COM3)*'
$port | Disable-PnpDevice -Confirm:$false
Start-Sleep -Seconds 3
$port | Enable-PnpDevice  -Confirm:$false

#  Remove ghost ports (non-present) to release their COM numbers. Preview first.
```

```
$ghosts = Get-PnpDevice -Class Ports | Where-Object { -not $_.Present }
$ghosts | Select-Object FriendlyName, InstanceId
$ghosts | ForEach-Object { pnputil /remove-device "$($_.InstanceId)" }

#  Install a staged vendor driver package and bind it to matching devices
pnputil /add-driver 'C:\Staging\VendorSerial\*.inf' /install
pnputil /scan-devices
```

**WATCH OUT · INSTANCE IDS CONTAIN AMPERSANDS AND BACKSLASHES**

Always quote the instance ID when passing it to `pnputil`, as above. In PowerShell, `Disable-PnpDevice` on a USB hub or controller will disconnect everything behind it, including the keyboard and mouse on some desktops. Target the *Ports* class device, never the hub.

## 5.3 Base template · scripted repair with evidence

This template wraps the inventory, the OS repair and the Windows Update reset into one parameterised script with `-WhatIf` support, so it can be peer-reviewed and run identically by any technician. It writes a JSON evidence file for the ticket.

REPAIR-USBSERIAL.PS1 · BASE TEMPLATE POWERSHELL

```
<#
.SYNOPSIS  Diagnose and repair USB serial (COM port) connectivity on a Windows 10/11 workstation.
.NOTES     Run elevated in Windows PowerShell 5.1 or PowerShell 7. Requires local administrator.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
  [string]   $ExpectedPort = 'COM3',             # port the instrument software uses
  [switch]   $RepairOs,                          # run DISM /RestoreHealth + SFC
  [switch]   $ResetWindowsUpdate,                # stop services, rename caches, restart
  [switch]   $RemoveGhostPorts,                  # uninstall non-present Ports devices
  [switch]   $DisableSelectiveSuspend,           # active power plan, AC and DC
  [string]   $EvidencePath = "$env:PUBLIC\Documents\UsbSerial-$(Get-Date -f yyyyMMdd-HHmm).json"
)
$ErrorActionPreference = 'Stop'
$log = [ordered]@{ Computer = $env:COMPUTERNAME; Operator = "$env:USERDOMAIN\$env:USERNAME"; StartUtc =
(Get-Date).ToUniversalTime() }

# ---- Stage 1: inventory (read-only) ---------------------------------------
$ports = Get-PnpDevice -Class Ports | Select-Object FriendlyName, Present, Status,
ConfigManagerErrorCode, InstanceId
$log.PortsBefore  = $ports
$log.ExpectedSeen = [bool]([System.IO.Ports.SerialPort]::GetPortNames() -contains $ExpectedPort)

# ---- Stage 2: OS integrity ------------------------------------------------
if ($RepairOs -and $PSCmdlet.ShouldProcess('Component store and system files', 'DISM /RestoreHealth +
SFC')) {
  & dism.exe /Online /Cleanup-Image /RestoreHealth | Out-Null; $log.DismExit = $LASTEXITCODE
  & sfc.exe /scannow | Out-Null;                               $log.SfcExit  = $LASTEXITCODE
}

# ---- Stage 3: Windows Update component reset ------------------------------
if ($ResetWindowsUpdate -and $PSCmdlet.ShouldProcess('SoftwareDistribution and catroot2', 'Rename
caches')) {
```

```
  $svc = 'wuauserv','cryptSvc','bits','msiserver'
  Stop-Service -Name $svc -Force
  $stamp = Get-Date -Format yyyyMMddHHmm
  Rename-Item "$env:windir\SoftwareDistribution"   "SoftwareDistribution.old-$stamp"
  Rename-Item "$env:windir\System32\catroot2"      "catroot2.old-$stamp"
  Start-Service -Name $svc
  $log.WuReset = $stamp
}

# ---- Stage 4: ghost ports ------------------------------------------------
if ($RemoveGhostPorts) {
  foreach ($g in (Get-PnpDevice -Class Ports | Where-Object { -not $_.Present })) {
    if ($PSCmdlet.ShouldProcess($g.FriendlyName, 'pnputil /remove-device')) {
      & pnputil.exe /remove-device "$($g.InstanceId)" | Out-Null
      $log.RemovedGhosts += @($g.FriendlyName)
    }
  }
}

# ---- Stage 5: USB selective suspend (active scheme) ---------------------
if ($DisableSelectiveSuspend -and $PSCmdlet.ShouldProcess('Active power plan', 'USB selective suspend =
Disabled')) {
  $sub = '2a737441-1930-4402-8d77-b2bebba308a3'; $set = '48e6b7a6-50f5-4782-a5d4-53bb8f07e226'
  & powercfg.exe /setacvalueindex SCHEME_CURRENT $sub $set 0
  & powercfg.exe /setdcvalueindex SCHEME_CURRENT $sub $set 0
  & powercfg.exe /setactive SCHEME_CURRENT
  $log.SelectiveSuspend = 'Disabled'
}

# ---- Stage 6: rescan and evidence ----------------------------------------
& pnputil.exe /scan-devices | Out-Null
Start-Sleep -Seconds 5
$log.PortsAfter  = Get-PnpDevice -Class Ports -PresentOnly | Select-Object FriendlyName, Status,
ConfigManagerErrorCode
$log.ExpectedNow = [bool]([System.IO.Ports.SerialPort]::GetPortNames() -contains $ExpectedPort)
$log.EndUtc      = (Get-Date).ToUniversalTime()
$log | ConvertTo-Json -Depth 4 | Out-File -FilePath $EvidencePath -Encoding utf8
Write-Host "Evidence written to $EvidencePath. $ExpectedPort present: $($log.ExpectedNow)"
```

### EXAMPLE · RUNNING THE TEMPLATE

INVOCATION POWERSHELL

```
#  Preview everything, change nothing
.\Repair-UsbSerial.ps1 -ExpectedPort COM3 -RepairOs -ResetWindowsUpdate -RemoveGhostPorts -WhatIf
#  Typical "drops after idle" fix
.\Repair-UsbSerial.ps1 -ExpectedPort COM4 -RemoveGhostPorts -DisableSelectiveSuspend
```

CHAPTER 6

# Methods D and E · Power Management and Enterprise Options

If the port works after a reboot but stops responding after the PC has been idle, asleep or on battery, disable USB selective suspend. There are three levels, from narrowest to broadest. Prefer the narrowest one that solves the problem.

## 6.1 Method D · Per device: Device Manager

1. In Device Manager expand **Universal Serial Bus controllers**.
2. For the hub the adapter is attached to (and for the adapter's own USB entry, if it has a **Power Management** tab), open **Properties › Power Management**.
3. Clear **Allow the computer to turn off this device to save power** and select **OK**.
4. Some vendor drivers add their own setting. For example, FTDI exposes *Enable Selective Suspend* under **Port Settings › Advanced...**.

## 6.2 Method D · Power plan: powercfg

The *USB selective suspend setting* lives in the power plan under the USB settings subgroup. Set it to 0 (Disabled) for both AC (plugged in) and DC (battery), then re-apply the scheme so it takes effect immediately.

ELEVATED COMMAND PROMPT · POWER PLAN CMD

```
:: Subgroup: USB settings  /  Setting: USB selective suspend setting
powercfg /setacvalueindex SCHEME_CURRENT 2a737441-1930-4402-8d77-b2bebba308a3 48e6b7a6-50f5-4782-a5d4-53bb8f07e226 0
powercfg /setdcvalueindex SCHEME_CURRENT 2a737441-1930-4402-8d77-b2bebba308a3 48e6b7a6-50f5-4782-a5d4-53bb8f07e226 0
powercfg /setactive SCHEME_CURRENT

:: Verify
powercfg /query SCHEME_CURRENT 2a737441-1930-4402-8d77-b2bebba308a3 48e6b7a6-50f5-4782-a5d4-53bb8f07e226
```

**NOTE · HIDDEN SETTING ON MODERN STANDBY DEVICES**

On some laptops the setting is hidden from the **Control Panel › Power Options › Change advanced power settings** dialog. `powercfg /attributes 2a737441-1930-4402-8d77-b2bebba308a3 48e6b7a6-50f5-4782-a5d4-53bb8f07e226 -ATTRIB_HIDE` makes it visible. If a power plan is enforced by Group Policy or Intune, change it there, or your local change will be overwritten.

## 6.3 Method D · Machine-wide: registry

For dedicated instrument PCs that should never suspend USB, a machine-wide value disables selective suspend in the USB stack regardless of the power plan. Use it only on dedicated, mains-powered workstations, and record it in the build documentation.

ELEVATED COMMAND PROMPT · REGISTRY (REBOOT REQUIRED) CMD

```
reg add HKLM\SYSTEM\CurrentControlSet\Services\USB /v DisableSelectiveSuspend /t REG_DWORD /d 1 /f
:: Roll back
reg delete HKLM\SYSTEM\CurrentControlSet\Services\USB /v DisableSelectiveSuspend /f
```

**WHY · NOT JUST "TURN OFF POWER SAVING EVERYWHERE"**

Disabling selective suspend raises idle power use and can shorten laptop battery life noticeably. It is the right fix for an instrument workstation and the wrong default for a fleet of mobile laptops. Scope the change with Intune or Group Policy targeting so it lands only on the devices that need it.

## 6.4 Method E · Remote PowerShell for one or a few PCs

When the instrument PC is on the corporate network and WinRM is enabled, you can run Method C without walking to the lab. Copy the template to the target or pass its contents as a script block.

REMOTE INVENTORY AND TARGETED FIX POWERSHELL

```
$pcs = 'PC-LAB-0001','PC-LAB-0002'
Invoke-Command -ComputerName $pcs -ScriptBlock {
  Get-PnpDevice -Class Ports | Select-Object FriendlyName, Present, ConfigManagerErrorCode
} | Sort-Object PSComputerName | Format-Table PSComputerName, FriendlyName, Present,
ConfigManagerErrorCode

#  Run the template remotely (positional argument = ExpectedPort)
Invoke-Command -ComputerName PC-LAB-0001 -FilePath .\Repair-UsbSerial.ps1 -ArgumentList 'COM3'
```

**NOTE · REMOTE SESSIONS AND THE CONSOLE**

You can inspect and repair drivers remotely, but you cannot unplug a cable remotely. Coordinate with someone at the instrument for swap tests, and warn the operator before disabling a port they may be using.

## 6.5 Method E · Intune Remediations for a fleet

**Remediations** (formerly Proactive remediations) pair a *detection* script with a *remediation* script. Intune runs detection on a schedule; if it exits with code 1, the remediation runs and the result is reported per device. Create them in **Intune admin center › Devices › Scripts and**

**remediations › Remediations › + Create**, run them in 64-bit PowerShell as SYSTEM, and assign them to a device group containing only instrument workstations.

DETECT-USBSERIAL.PS1 · DETECTION POWERSHELL

```powershell
#  Exit 1 = issue found (remediate). Exit 0 = compliant.
$problem = Get-PnpDevice -Class Ports -PresentOnly -ErrorAction SilentlyContinue |
           Where-Object ConfigManagerErrorCode -ne 0
$q = powercfg /query SCHEME_CURRENT 2a737441-1930-4402-8d77-b2bebba308a3 48e6b7a6-50f5-4782-a5d4-53bb8f07e226
$suspendOn = ($q | Select-String 'Current AC Power Setting Index: 0x00000001') -ne $null   # English output
if ($problem -or $suspendOn) {
  Write-Output "Ports with problems: $(@($problem).Count); selective suspend on AC: $suspendOn"; exit 1
}
Write-Output 'Serial ports healthy'; exit 0
```

REMEDIATE-USBSERIAL.PS1 · REMEDIATION POWERSHELL

```powershell
$sub = '2a737441-1930-4402-8d77-b2bebba308a3'; $set = '48e6b7a6-50f5-4782-a5d4-53bb8f07e226'
powercfg /setacvalueindex SCHEME_CURRENT $sub $set 0
powercfg /setdcvalueindex SCHEME_CURRENT $sub $set 0
powercfg /setactive SCHEME_CURRENT
Get-PnpDevice -Class Ports -PresentOnly | Where-Object ConfigManagerErrorCode -ne 0 | ForEach-Object {
  $_ | Disable-PnpDevice -Confirm:$false; Start-Sleep 2; $_ | Enable-PnpDevice -Confirm:$false
}
pnputil /scan-devices | Out-Null
Write-Output 'Selective suspend disabled; problem ports restarted'; exit 0
```

**WATCH OUT · LANGUAGE-DEPENDENT OUTPUT**

The detection script parses `powercfg` text, which is localised. On non-English builds, read the value from the power scheme registry key instead, or have the remediation always enforce the setting (it is idempotent).

## 6.6 Method E · Deploying the right driver centrally

- **Intune Win32 app**: package the vendor driver with an install command of `pnputil /add-driver .\*.inf /install` and a detection rule on the driver package version.
- **Windows driver update policies** (Windows Autopatch / Intune driver updates): approve or pause specific driver versions from Windows Update so a new release does not break a validated instrument.
- **Group Policy**: *Allow non-administrators to install drivers for these device setup classes* with the Ports class GUID `{4d36e978-e325-11ce-bfc1-08002be10318}` lets lab users plug in approved adapters without a ticket, while Device Installation Restrictions block everything else.

CHAPTER 7

# Verification, Rollback and Troubleshooting

## 7.1 Verification checklist

**Windows side**

- [ ] The adapter appears under **Ports (COM & LPT)** with no problem code
- [ ] The COM number matches the instrument software
- [ ] No ghost ports hold that number (**Show hidden devices**)
- [ ] The port is listed in `HKLM\HARDWARE\DEVICEMAP\SERIALCOMM`
- [ ] DISM and SFC report no unrepaired corruption
- [ ] No new errors under Kernel-PnP or System logs

**Instrument side**

- [ ] The software connects and reads data
- [ ] A test transfer or self-test completes
- [ ] Connection survives an unplug/replug
- [ ] Connection survives 30+ minutes idle, or sleep and resume
- [ ] Connection survives a full restart
- [ ] The user confirms normal operation

## 7.2 A quick loopback test

To prove the PC side independently of the instrument, short pins 2 (RX) and 3 (TX) on the adapter's DB9 connector with a loopback plug or a paper clip, then send text: anything written should be read back.

LOOPBACK TEST (PINS 2 AND 3 BRIDGED) POWERSHELL

```powershell
$sp = New-Object System.IO.Ports.SerialPort 'COM3', 9600, 'None', 8, 'One'
$sp.ReadTimeout = 2000; $sp.Open()
$sp.WriteLine('LOOPBACK-OK'); Start-Sleep -Milliseconds 200
$sp.ReadExisting(); $sp.Close()
```

## 7.3 Evidence for the ticket

- The JSON evidence file from the template (ports before/after, actions taken, operator, timestamps).
- Driver provider and version now installed, and the COM mapping.
- Whether DISM/SFC found and repaired corruption (from `CBS.log` / `dism.log`).
- Any power-management change made, and its scope (device, plan, machine-wide).

## 7.4 Rollback by action

| Action taken | How to undo it |
|---|---|
| **Driver updated** | Driver › Roll Back Driver, or `pnputil /delete-driver oemNN.inf /uninstall` then reinstall the previous package. |
| **Ghost device removed** | Nothing to restore: reconnecting that hardware recreates it. Reassign its COM number if needed. |
| **COM number reassigned** | Set it back in Port Settings › Advanced...; record both numbers in the ticket. |
| **Windows Update reset** | Stop the four services, delete the new folders, rename the `.old` folders back, start the services. |
| **Selective suspend disabled** | Re-run `powercfg` with value 1, re-tick the Device Manager box, or delete the `DisableSelectiveSuspend` value and reboot. |

## 7.5 Troubleshooting flowchart

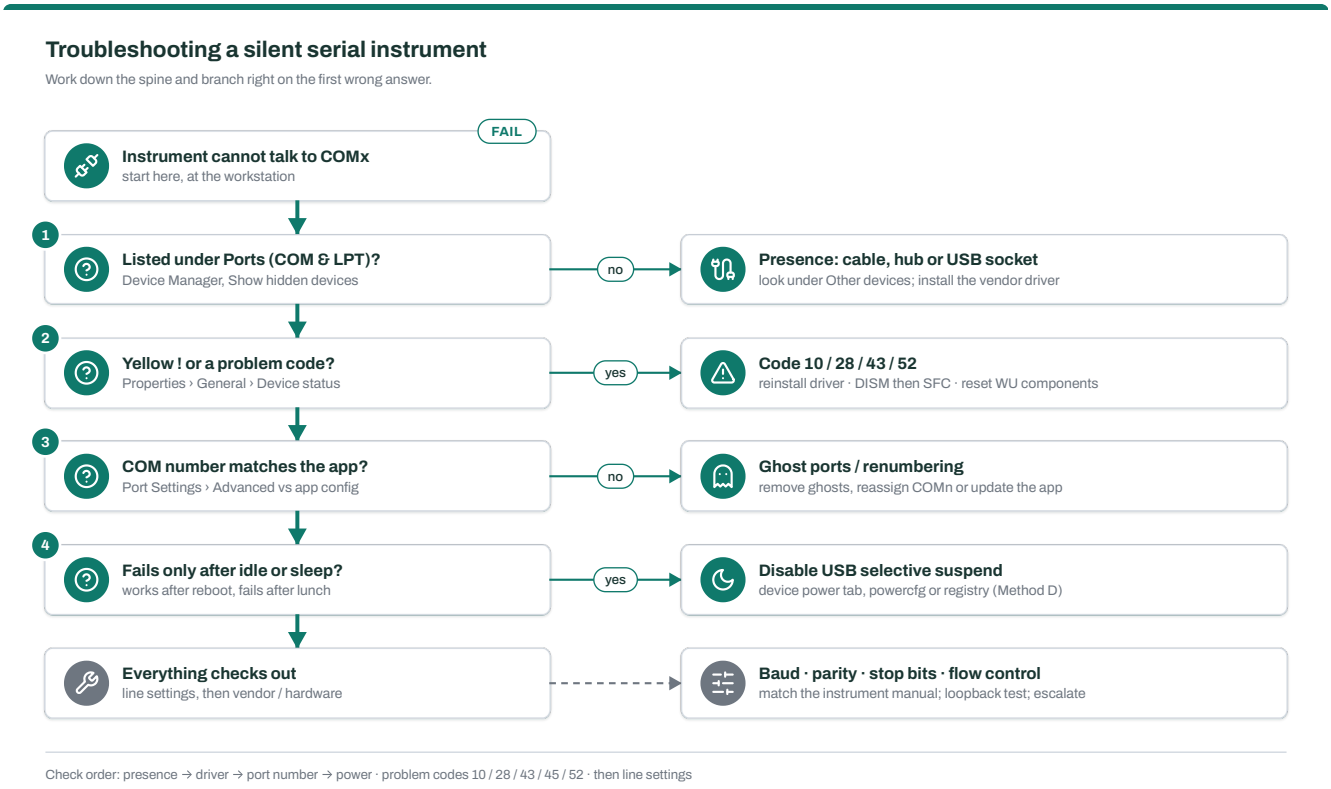



**Figure 7.** Check presence, driver, port number and power in that order, because each check depends on the one before it.

## 7.6 Symptom, cause and fix

| Symptom | Likely cause and fix |
|---|---|
| **Nothing changes in Device Manager when plugging in** | Cable, port or adapter failure; front-panel or hub port without enough power. Use a rear motherboard port, another cable, a spare adapter. |
| **Device under *Other devices*, Code 28** | No driver. Install the vendor package matching the VID. |
| **Code 10 immediately after Windows Update** | New driver rejects a clone chip, or a regression. Roll back; replace the adapter with a genuine one; pause that driver in Intune driver policy. |
| **Code 52, or installs stall** | Catalog or store damage. Method B (DISM → SFC → WU reset), then reinstall. |
| **Port number changes every time** | Adapter has no serial number and is moved between sockets. Label the socket, or use an adapter with a unique serial. |
| **"COM port in use" when reassigning** | Ghost reservation. Show hidden devices, uninstall the ghost, retry. |
| **Works after reboot, fails after idle** | USB selective suspend. Method D. |
| **"Access denied" opening the port in software** | Another process holds it (a second instance, a vendor utility, a modem manager). Close it or reboot; `handle.exe` from Sysinternals can find the holder. |
| **Data is garbled** | Baud, parity or flow-control mismatch, or wrong RS-232/RS-485 wiring. Fix in the application and cabling, not in Windows. |
| **Device installation blocked by policy** | Device Installation Restrictions are in force. Request an allow-listing of the hardware ID or class from the policy owner. |

## 7.7 When to escalate

Escalate to the instrument vendor or hardware team when a known-good adapter and cable work on another PC but not this one after Methods A–D, when the adapter is built into the instrument (no swap possible), or when a validated system's driver version must change. On regulated instrument PCs, a driver change may itself need a change record and revalidation.

CHAPTER 8

# Summary, Glossary and Review

## 8.1 Chapter summary

- A COM port is a name assigned to a USB serial function driver by the COM Name Arbiter. Most failures are naming, driver or power problems, not broken hardware.
- Device Manager problem codes point to the layer: 28 missing driver, 10/43 start or device failure, 52 signature verification, 45 not connected.
- Repair the OS with DISM first, then SFC. Reset SoftwareDistribution and catroot2 when driver installs fail or signatures will not verify.
- Disable USB selective suspend at the narrowest scope that fixes "fails after idle", and use Remediations to keep instrument PCs in that state. Remove ghost ports so COM numbers stay predictable.

**Check your understanding**

1. An adapter receives COM5 when plugged into the left USB socket and COM3 in the right. What does this tell you about its chip, and how do you make the instrument software reliable?
2. Why should `DISM /RestoreHealth` run before `sfc /scannow`?
3. Which two services must be stopped before renaming `SoftwareDistribution` and `catroot2`, and why does catroot2 matter for drivers?
4. A port works every morning but fails after lunch. Which method do you apply first, at which scope, and why not machine-wide on every laptop?
5. Write the PowerShell pipeline that lists non-present serial ports and the command that removes one.

## 8.2 Glossary

| | |
|---|---|
| **COM port** | Windows name (COM1–COM256) for a serial communications device. |
| **ComDB** | COM Name Arbiter bitmap of reserved COM numbers; non-present devices that keep a reservation are "ghost ports". |
| **SFC** | System File Checker; restores protected system files from the component store. |
| **catroot2** | Catalog database used by Cryptographic Services to verify driver signatures. |
| **Selective suspend** | USB power feature that suspends idle devices individually. |